C语言结构体

掌握复杂数据类型，构建复杂数据结构的基础

1. 什么是结构体

结构体 (struct) 是一种用户自定义的复合数据类型（有些书上也叫作：聚合数据类型），允许将多个不同类型的数据组织成一个逻辑单元的数据结构。可以将结构体看作是一个包含了一组变量的集合，这些变量被称为结构体的**成员** (members) **或字段** (fields)。

1.1 结构的声明

语法格式：

```
struct tag
{
    member-list;
}variable-list;
```

//tag是结构体的名称标签

你如描述一个学生：

```
struct Stu
{
    char name[20];//名字
    int age;//年龄
    char sex[5];//性别
    char id[20];//学号
};//分号不能丢
```

1.2 结构体变量的创建和初始化

```
#include <stdio.h>
struct Stu
{
    char name[20];//名字
    int age;//年龄
    char sex[5];//性别
    char id[20];//学号
};

int main()
{
    //按照结构体类型创建结构体变量
    struct Stu s = {"张三", 20, "男", "20230101001" };
    printf("name: %s\n", s.name);
    printf("age: %d\n", s.age);
    printf("sex: %s\n", s.sex);
    printf("id : %s\n", s.id);

    //匿名结构体类型变量定义和初始化-不推荐用
    struct Stu s2 = { .age = 18, .name = "Lisa", .id = "20230101002", .sex = "女" };
    printf("name: %s\n", s2.name);
    printf("age: %d\n", s2.age);
    printf("sex: %s\n", s2.sex);
    printf("id : %s\n", s2.id);
    return 0;
}
```

1.3 结构的特殊声明

在声明结构的时候，可以不完全的声明。比如：

```
//匿名结构体类型
struct
{
    int a;
    char b;
    float c;
}*p;

struct
{
    int a;
    char b;
    float c;
}s[20], *q;
```

上面的两个结构的声明时省略掉了结构体标签（tag），那么问题来了？

思考：下面的代码合法吗？

```
p = &s;
```

虽然这个结构体的创建语法一样，但编辑器会把上面的两个声明当成完全不同，毕竟编译的时候两个类型，此时这2个结构体的内存是完全独立的。

1.4 结构的自引用

在结构中包含一个类型为该结构本身的成员是否可以呢？比如，定义一个链表的节点：

```
struct Node
{
    int data;
    struct Node next;
};
```

上述代码正确吗？如果正确，那sizeof(struct Node)是多少？

计算结果，就是报错的，因为一个结构体中包含一个同类型的结构体变量，这样结构体变量就无小最会无穷的大，是不合理的。

正确的自引用方式：

```
struct Node
{
    int data;
    struct Node* next;
};
```

在结构体自引用使用的过程中，关键字typedef对重名结构体类型重命名，也容易引入问题，看看下面的代码，可行吗？

```
typedef struct
{
    int data;
    Node* next;
}Node;
```

答案是不行的，因为Node是对前面的匿名结构体类型的重命名产生的，但是在匿名的结构体内部因即使用Node类型本质结构体类型，这是不行的。

解决方案如下：定义结构体不要使用匿名结构体了。

```
typedef struct LinkNode
{
    int data;
    struct LinkNode* next;
}Node;
```

2. 结构体内存对齐

内存对齐是计算机硬件（特别是CPU）对内存中数据存储位置的一种要求。现的语言数据的结构内存地址必须是其自身大小的整数倍。

2.1 对齐规则

首先得掌握结构体的对齐规则：

1. 结构体的第一个成员对齐到结构体变量起始位置的偏移量为0的地址处。
2. 从第二个成员变量开始，都要对齐到某个对齐数的整数倍的地址。
3. 结构体变量的总大小应该是所有 **对齐数** 中最大对齐数的整数倍。
   - VS 中默认的对齐数为 8
   - Linux中gcc没有默认对齐数，对齐数就是成员自身的大小
4. 结构体内嵌套了其他结构体，结构体中嵌套的结构体成员从它的最大对齐数开始，所有对齐数中最大的，对齐数的整数倍。
5. 结果是要了解结构体的内部成员，就要清楚结构体成员对齐规则以及结构体中最大对齐数的整数倍，结构体的整体大小就是所有最大对齐数（含嵌套结构体中成员的对齐数）的整数倍。

练习1

```
struct S1
{
    char c1;
    int i;
    char c2;
};
printf("%d\n", sizeof(struct S1));
```

练习2

```
struct S2
{
    char c1;
    char c2;
    int i;
};
printf("%d\n", sizeof(struct S2));
```

练习3

```
struct S3
{
    double d;
    char c;
    int i;
};
printf("%d\n", sizeof(struct S3));
```

练习4-结构体嵌套问题

```
struct S4
{
    char c1;
    struct S3 s3;
    double d;
};
printf("%d\n", sizeof(struct S4));
```

2.2 为什么存在内存对齐？

内存对齐的存在主要是为了提升内存访问速度和保证硬件平台。

1. **提升 CPU 读取效率（性能原因）**
   - 这是最核心的原因。CPU 访问内存不是逐字节读取的，而是以字 (Word) 为单位进行读取。一个现代大小通常是 4 字节 (32位系统) 或 8 字节 (64位系统)。
   - **对齐的结果：**如果一个 4 字节的 int 数据存储在一个 4 的倍数的地址上（例如地址 0x0000），那么这个数据会完整地落在一个内存字中。CPU 只需要进行一次内存读取操作就可以得到这个完整的 int。
   - **未对齐的结果：**如果一个 int 数据一半在 0x0000 和 0x0004 的内存块中，而另一半在 0x0004 和 0x0008 的内存块中，CPU 为了读取这个数据，可能需要进行两次内存读取操作，然后将这两个部分拼接起来。这会显著降低性能。
2. **硬件平台要求（强制性原因）**
   - 某些硬件平台（如一些 RISC 架构的 CPU）不支持对非对齐的内存地址进行访问。在这些平台上，尝试访问一个未对齐的数据可能会导致硬件异常（例如总线错误 Bus Error）或程序崩溃。
   - 虽然现代 x86/x64 架构 CPU 功能强大得多，可以处理非对齐的访问，但通常会带来显著的性能损失。因为编译器通常会以最快的速度执行代码，因此，为了保证代码的可移植性和性能不稳定性，通常会对齐内存是必要的。
   - **通常内存对齐是强制性，可以通过编译指令或修改编译器来调整。**

```
struct S1
{
    char c1;
    int i;
    char c2;
};

struct S2
{
    char c1;
    char c2;
    int i;
};
```

S1和S2成员完全相同，但是S1和S2所占空间的大小有了一定区别。

2.3 修改默认对齐数

#pragma 这个预处理指令，可以改变编译器的默认对齐数。

```
#include <stdio.h>

#pragma pack(1)//设置默认对齐数为1
struct S
{
    char c1;
    int i;
    char c2;
};
#pragma pack()//取消设置的对齐数，还原为默认

int main()
{
    //输出一下现在的大小
    printf("%d\n", sizeof(struct S));
    return 0;
}
```

结构体在对齐方式不合适的时候，我们可以自己更改默认对齐数。

3. 结构体传参

```
struct S
{
    int data[1000];
    int num;
};

struct S s = {{1,2,3,4}, 1000};

//结构体传参
void print1(struct S s)
{
    printf("%d\n", s.num);
}

//结构体地址传参
void print2(struct S* ps)
{
    printf("%d\n", ps->num);
}

int main()
{
    //打印结构体数据
    print1(s);//传结构体
    print2(&s);//传地址
    return 0;
}
```

上面的print1和print2函数哪个好些？

答案是：首选print2函数。

原因：函数在调用时，会创建形参副本。会导致时间和空间上的系统开销。如果传递一个结构体对象的时候，结构体过大，参数压栈的时间和内存消耗比较大，所以会导致性能的下降。

结论：结构体传参的时候，最终结构体地址。

本节总结

- 结构体是一种将不同类型数据组织成一起的复合数据类型。
- 掌握结构体的声明、变量定义和初始化（包括匿名结构体和指定成员初始化）。
- 理解结构体内存对齐的规则及其背后的原因。
- 掌握结构体正确的自引用方法（使用指针）。
- 深入理解结构体内存对齐的方式（编译器、架构、性能）。
- 了解为何需要进行内存对齐（提升速度、兼容硬件）。
- 学会使用 #pragma pack() 修改默认对齐数。
- 理解结构体传参时，传地址（指针）比直接传递的效率更高。